

Algorithms Programming Project I

Greedy Algorithms

Team members

Divya Upadhyay - 29228848

Contributions - Prepared report, worked on strategy 1 and 3 and code for bonus part

Raj Vora - 35551411

Contributions - Prepared report, worked on strategy 2 and 4 and code for bonus part

Greedy Strategies

Strategy 1

Algorithm:

Inputs:

n: number of days to paint the houses

m: number of houses available to be painted

houses: a list of tuples containing the start day and end day of each house

Outputs:

houses_painted: a list of integers containing the positions of the painted houses

Steps:

Initialize an empty priority queue, pq, to store the houses in ascending order based on their start day.

Initialize an empty list, houses_painted, to store the positions of the painted houses.

Initialize an integer variable, location, to keep track of the location of each house in the input list.

Initialize two integer variables, day and ptr, to keep track of the current day and the current index of the input list, respectively.

For each house in the input list, add it to the priority queue with its start day and end day as a tuple, along with its location in the input list as a secondary priority.

While there are still houses in the priority queue and there are still days left to paint houses:

a. Get the start day and end day of the first house in the priority queue and its position in the input list.

b. If the start day of the first house in the input list is greater than the current day, increment the current day and continue the loop.

c. If the start day of the first house in the priority queue is less than or equal to the current day and the end day of the first house in the priority queue is greater than or equal to the current day, append the position of the first house in the input list to `houses_painted`.

d. Increment the current day by 1.

e. Set the current index of the input list to the position of the last painted house in `houses_painted`.

return `houses_painted`.

Instance where strategy leads to optimal solution:

```
(venv) D:\2023\UF\AoA\Project1\greedy_algo_project1_aoa_cot5405>python Strategy_1.py
3 3
1 3
2 4
3 5
1 2 3
```

Instance where strategy leads to non-optimal solution:

```
(venv) D:\2023\UF\AoA\Project1\greedy_algo_project1_aoa_cot5405>python Strategy_1.py
5 4
1 2
1 2
2 3
2 3
1 2 3
```

Reason for non-optimality: Choosing houses with earlier start date may lead to skipping of house on later as they will no longer be available past their endDay.

Time Complexity Analysis:

The time complexity of the code is $O(m \log m)$, where m is the number of houses. This is because the first loop that builds the priority queue takes $O(m \log m)$ time, and the second loop that pops houses from the priority queue takes $O(n \log m)$ time (since it may iterate up to n times and popping from the heap takes $\log m$ time in the worst case).

Therefore, the overall time complexity is dominated by the first loop and the code is in $O(m \log m)$ time complexity. Additionally, the time complexity of the code is also in $\Omega(m \log m)$, since we are pushing m elements into a heap, and each push takes $\log m$ time in the worst case. This means that the code is both upper and lower bounded by $m \log m$.

The time complexity of the code can be expressed as $\Theta(m \log m)$. Since in the problem the houses are already inputted in sorted order of startDay and popping such houses from heap will result in $O(1)$ time complexity, so we can conclude the overall time complexity of the code will be $\Theta(n)$.

Strategy 2

Algorithm:

Sort the houses based on their start day, and end day (if start day is same), in ascending order.

1. Initialize a priority queue (pq), a list to store painted houses (houses_painted), and a variable to keep track of the current location (location).
2. For each day from 1 to n, do the following:
 - a. Add the houses that are available to be painted on the current day to the priority queue.
 - b. Paint the house that has the earliest end day in the priority queue, as long as its end day is not earlier than the current day. If there are no such houses, move on to the next day.
 - c. Add the painted house's location to the houses_painted list.
3. Return the houses_painted list.

Instance where strategy leads to optimal solution

```
~/projects/aoa/greedy algo project1 aoa cot5405 > dev-dv ± python3 Strategy_2.py
5 4
1 4
2 5
3 6
4 7
1 2 3 4
```

Instance where strategy leads non-optimal solution

```
~/projects/aoa/greedy algo project1 aoa cot5405 > dev-dv ± python3 Strategy_2.py
5 4
1 3
2 3
2 3
3 5
1 2 4
```

It would be optimal to paint in order of 1 2 3 4

Time Complexity analysis

Building the heap takes $O(\log m)$ time in the worst case, since it may need to add all m houses to the heap. Painting the house takes $O(1)$ time. Therefore, the time complexity of the loop is $O(n \log m)$ in the worst case, since there are n days and for each day, we may need to build a heap of up to m houses.

In total, the time complexity of the function is $O(m \log m + n \log m)$ in the worst case. However, in practice, the number of available houses on each day is typically much smaller than m , so the actual running time of the function may be much faster.

Therefore, we can say that the big theta time complexity of the `max_houses` function is $\Theta(n+m \log m)$.

Strategy 3

Algorithm:

Sort the houses based on the startDay and endDay (if startDay is same) in ascending order.
Initialize an empty heap 'pq' to store the houses available for painting, and an empty list 'houses_painted' to store the houses painted.
Set location to 0.

For each day in the range 1 to n:

- a. While there are unpainted houses and the startDay of the first house in the 'houses' list is less than or equal to the current day:
 - i. Remove the first house from the 'houses' list.
 - ii. Increment 'location'.
 - iii. Calculate the duration of the house (endDay - startDay + 1).
 - iv. Push the house into the 'pq' heap with the duration as the priority.
- b. While there are houses in the 'pq' heap:
 - i. Pop the house with the shortest duration.
 - ii. If the endDay of the house is greater than or equal to the current day, append the house's location to the 'houses_painted' list.

Return the 'houses_painted' list.

The time complexity of the algorithm is $O(n + m \log(m))$.

Instance where strategy leads to optimal solution:

```
(venv) D:\2023\UF\AoA\Project1\greedy_algo_project1_aoa_cot5405>python Strategy_3.py
4 3
1 2
2 3
3 4
1 2 3
```

Instance where strategy leads to non-optimal solution:

```
(venv) D:\2023\UF\AoA\Project1\greedy_algo_project1_aoa_cot5405>python Strategy_3.py
3 4
1 2
1 4
2 3
3 4
1 3 4
```

Reason for non-optimality: Even though the house 2 was available for the longer duration it was not picked-up since the house which was recently published has a shorter duration.

Time Complexity Analysis:

The algorithm has a single for loop that iterates over each day from 1 to n , resulting in a time complexity of $O(n)$.

For each day, the algorithm builds a heap of houses that are available to be painted on that day. The size of the heap can be at most m since all the houses could be available on the same day. Building the heap takes $O(m)$ time.

Then, the algorithm repeatedly pops the house with the shortest duration from the heap until it finds a house that is available on the current day. The popping operation takes $O(\log(m))$ time since the heap has a maximum size of m . Since the heap is built once for each day, the total time spent in popping operations is $O(m \log(m))$. The time complexity of the above algorithm is $O(n + m \log(m))$ in the worst-case, which is also the best-case, and the average-case scenario. Therefore, the big-oh and theta time complexity of the algorithm is $\Theta(n + m \log(m))$.

Strategy 4

Algorithm:

Sort the list of houses based on startDay and endDay (if startDay is same).

1. Initialize a priority queue (pq), a list to store painted houses (houses_painted), and a variable to keep track of the current location (location).
2. For each day from 1 to n, do the following:
 - a. Add the houses that are available to be painted on the current day to the priority queue.
 - b. Paint the house that has the earliest end day in the priority queue, as long as its end day is not earlier than the current day. If there are no such houses, move on to the next day.
 - c. Add the painted house's location to the houses_painted list.
3. Return the houses_painted list.

Instance where strategy leads to optimal solution

```
~/projects/aoa/greedy_algo/project1/aoa_cot5405 > dev-dv ± python3 Strategy_4.py
5 4
1 3
2 3
2 3
3 5
1 2 3 4
~/projects/aoa/greedy_algo/project1/aoa_cot5405 > dev-dv ± python3 Strategy_4.py
5 4
1 4
2 3
3 5
4 5
1 2 3 4
~/projects/aoa/greedy_algo/project1/aoa_cot5405 > dev-dv ±
```

Instance where strategy leads to non-optimal solution

As we saw above that the input for strategy 2 which lead to non optimal solution leads to optimal solution using strategy 4

Time Complexity analysis

The time complexity of the above algorithm is $O(n \log(m))$, where n is the number of days and m is the number of available houses.

The algorithm iterates through each day once, and for each day it performs two main operations:

1. Build the heap for the house endDay in the ascending order - this operation takes $O(m \log(m))$ time, as it involves inserting each house into a priority queue, which has a cost of $O(\log(m))$ per insertion, and there are m houses.
2. Paint the house that stops being available the earliest - this operation takes $O(\log(m))$ time, as it involves removing the minimum element from the priority queue, which has a cost of $O(\log(m))$.

Since the number of days is at most n and the total number of operations performed by the algorithm is $O(m \log(m))$, the overall time complexity of the algorithm is $O(n \log(m))$.

Therefore, the big theta time complexity of the above algorithm is also $O(n \log(m))$.

Proof of Correctness

Let H be the set of all houses, and let P be the set of painted houses that are chosen by the algorithm. We want to prove that P is the maximum set of non-overlapping houses that can be painted.

We can start by observing that sorting the houses by their start day and end day (if the start days are the same) ensures that the houses with earlier start days are considered first, which is optimal for minimizing the overlap between painted houses.

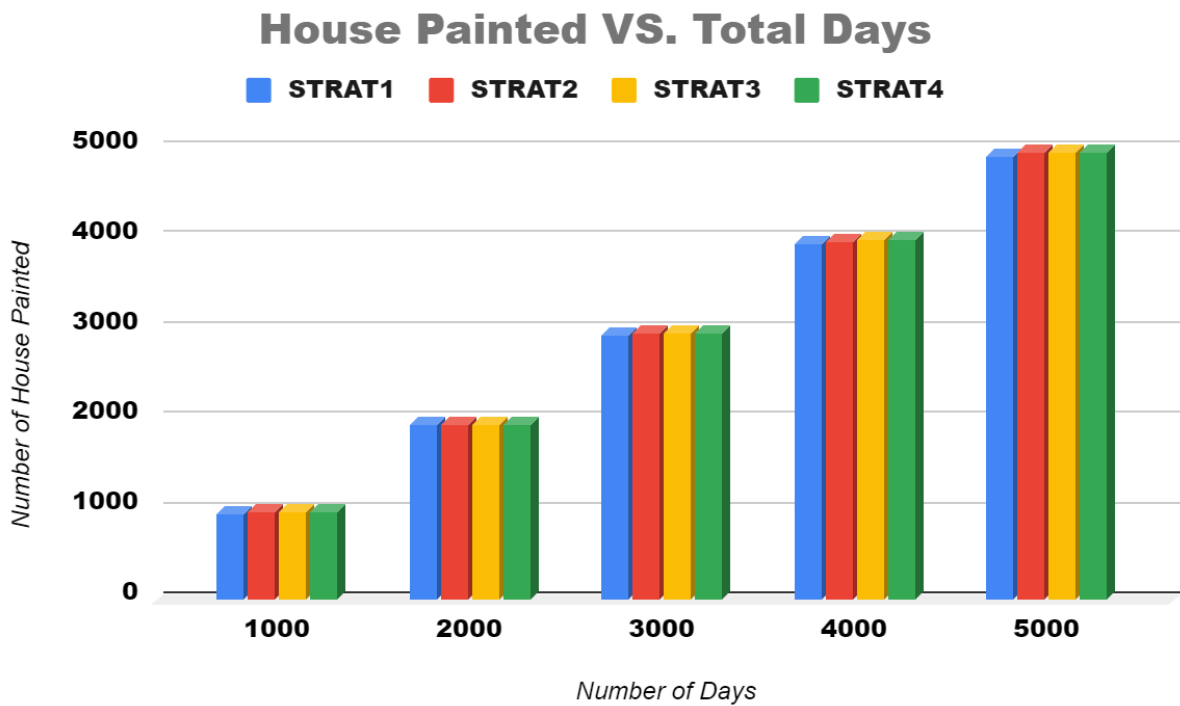
Next, we can prove that at each iteration of the algorithm, the house that stops being available the earliest is chosen. This is because the priority queue is sorted by the house's end day in ascending order, and the earliest available house is always at the top of the priority queue.

Finally, we can prove that the set of painted houses P is the maximum set of non-overlapping houses that can be painted. This is because if there exists a set Q of non-overlapping houses that can be painted, and $|Q| > |P|$, then there must exist a house h in Q that is not in P . However, since the algorithm chooses the house that stops being available the earliest at each iteration, and h is not in P , this means that h must stop being available later than some other house in P . Therefore, if h is painted, it would cause overlap with the other house that stops being available earlier, which contradicts the assumption that Q is a set of non-overlapping houses that can be painted.

Therefore, the algorithm correctly computes the maximum set of non-overlapping houses that can be painted, and the code implementation correctly implements the algorithm.

Experimental Comparative Study

Number of days	Strat1	Strat2	Strat3	Strat4
1000	961	974	977	977
2000	1937	1952	1952	1952
3000	2931	2947	2950	2950
4000	3937	3969	3985	3986
5000	4906	4952	4963	4963



Conclusion

After preparing the algorithms for different strategies we come to the conclusion that strategy 4 is the most optimal strategy. We can draw parallels with the interval scheduling problem discussed in class. When we started with the problem definition the obvious data structure to use was arrays. But then the hint suggested to use priority queues and the runtime of the codes drastically reduced. Problem faced in strategy 1 was to bring it down to $O(n)$ by using the while loop. Rest of the strategies were almost similar par the way we presented the queue to the algorithm. The bonus strategy is a more efficient version of strategy 4 in the way that we loop over the houses instead of the number of days the painter is available. We can also observe the improved runtime efficiency from the graph. We got to learn about how the problem needs to be approached and finding an optimal solution in terms of time complexity and then again improving the efficiency of that solution in terms of runtime.

Bonus

Algorithm:

Initialize an empty priority queue, pq, to store the houses in ascending order based on their end day.

Initialize the curr_day to 1 and houses_painted list to empty

Set i=0

While curr_day is less than or equal to n and houses list is not empty:

a. While i is less than the length of houses and start day of houses[i] is greater than curr_day:

i. Increment i by 1

b. If i is less than the length of houses:

i. Append the house index of first house in the priority queue to the houses_painted list

c. Increment curr_day by 1

Return houses_painted

Time Complexity Analysis

The sorting of the houses by end day takes $O(m \log m)$ time, and the heap operations for adding and removing elements take $O(\log m)$ time for each house. The while loop that iterates through each day from 1 to m takes $O(m)$ time, and the nested while loop that finds the first house that can be painted on the current day takes $O(m)$ time in the worst case. Therefore, the overall time complexity is dominated by the sorting and heap operations, which gives a time complexity of $O(m \log m)$.

Space Complexity Analysis

The space complexity of the given code is $O(m)$, where m is the number of houses.

The space used by the heap priority queue is proportional to the number of houses, which gives a space complexity of $O(m)$. The space used by the other variables and data structures in the code is constant and does not depend on the number of houses, so they do not contribute to the space complexity. Therefore, the overall space complexity is $O(m)$.

Proof of Correctness

Let H be the set of all houses, and let P be the set of painted houses that are chosen by the algorithm. We want to prove that P is the maximum set of non-overlapping houses that can be painted.

We can start by observing that sorting the houses by their end day ensures that the houses with earlier end days are considered first, which is optimal for minimizing the overlap between painted houses. This is because if a house with an earlier end day is painted, it frees up more time for painting other houses.

Next, we can prove that at each iteration of the algorithm, the first house that can be painted on the current day is chosen. This is because the sorted order of the houses ensures that the houses with earlier start days are considered first, and the heap priority queue ensures that the house with the lowest cost is chosen among the houses that can be painted on the current day. Therefore, the algorithm chooses the most cost-effective house that can be painted on each day.

Finally, we can prove that the set of painted houses P is the maximum set of non-overlapping houses that can be painted. This is because if there exists a set Q of non-overlapping houses that can be painted, and $|Q| > |P|$, then there must exist a house h in Q that is not in P . However, since the algorithm chooses the most cost-effective house that can be painted on each day, and h is not in P , this means that there must be another house in Q that is more cost-effective to paint than h on each day that h is painted. Therefore, the algorithm would have chosen that house instead of h , which contradicts the assumption that Q is a set of non-overlapping houses that can be painted.

Therefore, the algorithm correctly computes the maximum set of non-overlapping houses that can be painted, and the code implementation correctly implements the algorithm.

Runtime analysis

No. of days	Run time Strategy 4 (seconds)	Run time efficient version (seconds)
10000	0.0319	0.01296
25000	0.080426	0.0260415
50000	0.316822	0.0800793
75000	0.7000191	0.1360306
95000	1.2402286	0.1498150

Runtime Analysis

